TP1 — Configuration avancée du système Linux

Andrea Garcia Magnanon · BTC2 · 2 novembre 2026

1. Présentation du travail

Dans ce TP, j'ai repris le serveur de l'entreprise fictive Hélios Logistique. L'objectif était de rendre la collecte de télémétrie plus fiable, de maintenir la communication en cas de coupure d'un lien et d'automatiser l'archivage des données.

J'ai travaillé avec deux machines Rocky Linux 9.8 dans VirtualBox : **srv-core**, qui héberge les services Hélios, et **srv-edge**, qui me sert à vérifier le réseau. Les commandes d'administration ont été exécutées avec les droits root.

Réseau	srv-core	srv-edge
Métier	10.42.0.10/24 sur bond0	10.42.0.11/24 sur enp0s9
Transport — VLAN 42	10.42.42.10/24 sur bond0.42	10.42.42.11/24 sur enp0s9.42
Administration et Internet	Carte NAT enp0s3	Carte NAT enp0s3

Pour l'administration, j'ai utilisé les redirections SSH de VirtualBox : le port 2221 pour srv-core et le port 2222 pour srv-edge. Cette solution remplace le réseau host-only prévu pour l'accès d'administration, tout en conservant les interfaces métier attendues dans le TP.

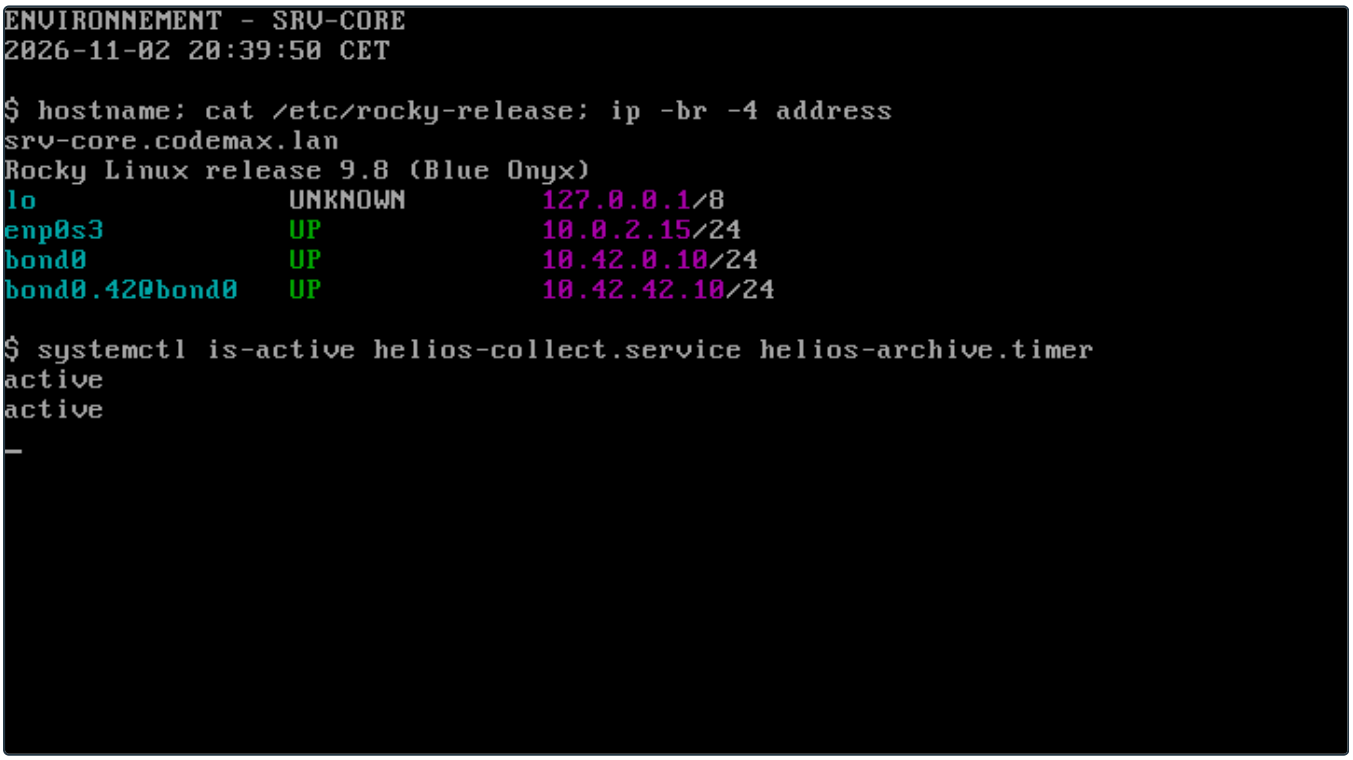


Figure 1 — Adresses de srv-core et état des services Hélios.

Les captures de ce rapport proviennent directement des consoles VirtualBox. Certaines montrent un contrôle en direct, d'autres affichent les résultats enregistrés pendant les tests ; leur titre indique la différence.

2. Mesurer et alléger le démarrage

J'ai commencé par comparer le démarrage avec et sans les optimisations demandées. Pour avoir une référence exploitable, j'ai réactivé firewalld et l'attente réseau, puis redémarré la VM. J'ai ensuite appliqué les réglages du TP et effectué un deuxième démarrage.

Mesure	Avant	Après
Noyau	2,380 s	2,194 s
Initrd	17,147 s	15,902 s
Espace utilisateur	31,106 s	23,845 s
Temps total	**50,634 s**	**41,943 s**

Sur ces deux essais, le gain est de **8,691 secondes**, soit environ **17,2 %**. Il s'agit d'une comparaison réalisée avec les services Hélios déjà installés, et non d'une mesure juste après l'installation de Rocky Linux. La charge de l'ordinateur et les caches peuvent aussi faire varier les temps.

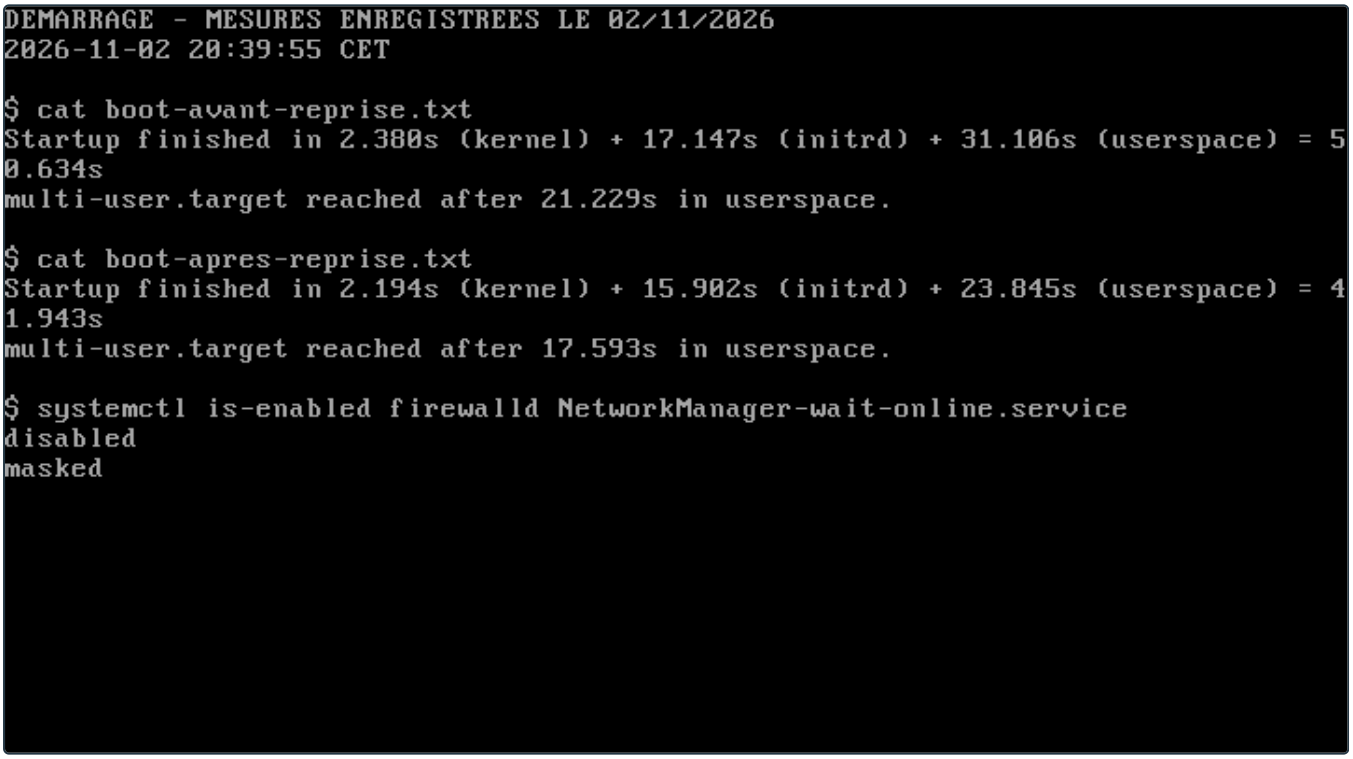


Figure 2 — Mesures de démarrage enregistrées et état final des services désactivés.

J'ai désactivé temporairement **firewalld**, comme demandé dans l'énoncé, qui prévoit sa réactivation au TP3. J'ai également masqué **NetworkManager-wait-online.service** pour supprimer l'attente globale du réseau. Le collecteur lit ici des données locales : il n'a pas besoin d'attendre un serveur distant pour fonctionner.

Dans les résultats de `systemd-analyze blame`, j'ai notamment relevé dracut-initqueue à 13,569 s et kdump à 12,558 s avant modification. Ces durées se chevauchent : on ne peut pas les additionner pour retrouver le temps total. J'ai donc conservé aussi la chaîne critique et les deux graphiques de démarrage, **boot-avant.svg** et **boot-apres.svg**.

3. Superviser et limiter le collecteur

Le script `helios-collect.sh` enregistre la date, la charge du système et la mémoire disponible dans un fichier CSV. Je l'ai fait fonctionner avec le compte système **helios**, plutôt qu'avec root.

Avec `Restart=on-failure` et `RestartSec=5s`, systemd doit le relancer s'il s'arrête anormalement. Pour le vérifier, j'ai envoyé un **kill -9** au processus principal. Le PID est passé de **2381 à 2387**, et le service est redevenu actif. Cela confirme que la relance ne dépend plus d'une intervention manuelle.

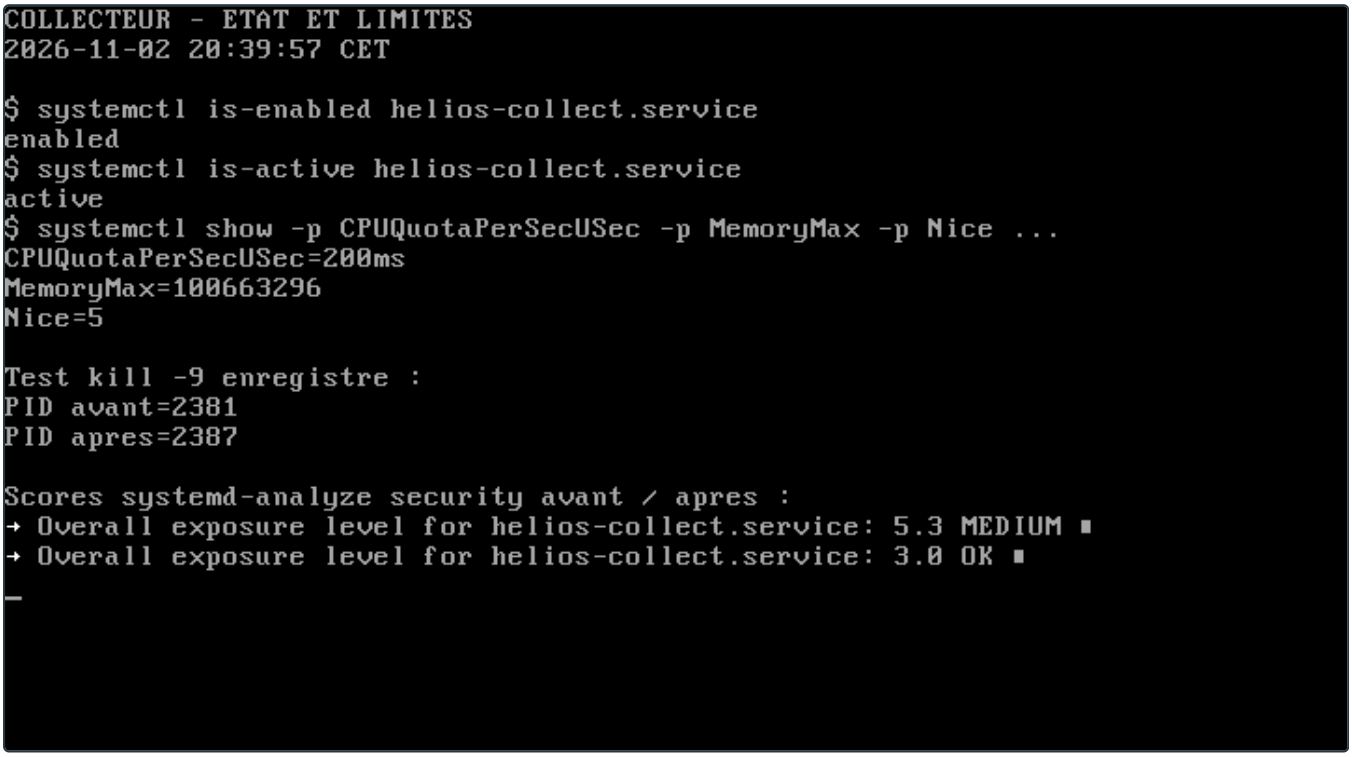


Figure 3 — Service actif, limites appliquées, changement de PID et scores de sécurité.

J'ai placé les réglages locaux dans **override.conf**, sans modifier l'unité de base : une collecte toutes les 5 secondes, un quota CPU de 20 %, une limite mémoire de 96 Mio, un poids E/S de 50 et une priorité Nice de 5. La valeur observée, **MemoryMax=100663296**, correspond bien à 96 Mio. Le quota CPU représente 20 % d'un processeur ; le poids E/S règle une priorité relative et ne garantit pas un débit disque.

J'ai ensuite renforcé l'isolation du service dans ce même fichier : masque de création des fichiers à 0077, protection des modules et journaux du noyau, interdiction des priorités temps réel, restrictions sur les espaces de noms et sur les sockets IP. Le collecteur continue de fonctionner avec ces protections. Le score d'exposition donné par systemd passe de **5,3 à 3,0** ; ce score reste un indicateur, pas une garantie de sécurité complète.

Enfin, j'ai rattaché le collecteur à **helios.target** et vérifié sa présence dans les dépendances. Une relation Wants démarre le service avec la cible, mais ne propage pas automatiquement son arrêt. Pour l'exercice sur les processus, j'ai lancé une charge à Nice 19, modifié sa priorité à 5, réglé sa priorité E/S en best-effort 7, puis arrêté le processus. Les sorties correspondantes sont jointes au rendu.

4. Vérifier la continuité du réseau

Pour éviter qu'une seule carte réseau devienne un point de panne, j'ai regroupé **enp0s9 et enp0s10** dans **bond0**, en mode **active-backup**. Une carte travaille pendant que l'autre reste disponible pour prendre le relais. Le TP utilise ce mode dans VirtualBox ; je n'ai pas mis en place de LACP.

J'ai choisi enp0s9 comme interface principale, avec un contrôle de lien toutes les 100 ms. Ensuite, j'ai lancé un ping continu vers srv-edge et coupé l'adaptateur 3 depuis l'hôte avec `VBoxManage controlvm srv-core setlinkstate3 off`.

```
BOND - PREUVES DU TEST DE COUPURE ENREGISTRE
2026-11-02 20:39:58 CET

Avant coupure :
Currently Active Slave: enp0s9
MII Status: up
MII Status: up
MII Status: up
Pendant la coupure de adaptateur 3 :
Currently Active Slave: enp0s10
MII Status: up
MII Status: up
MII Status: down
Apres retablissement :
Currently Active Slave: enp0s9
MII Status: up
MII Status: up
MII Status: up

Bilan du ping pendant le test :
--- statistiques ping 10.42.0.11 ---
100 paquets transmis, 98 reçus, 2% packet loss, time 20662ms
rtt min/avg/max/mdev = 0.870/3.934/64.282/6.583 ms
```

Figure 4 — État du bond avant, pendant et après la coupure, puis bilan du ping.

Pendant la coupure, **enp0s9 passe down et enp0s10 devient l'interface active**. Après rétablissement du lien, enp0s9 reprend son rôle principal et les deux ports sont à nouveau up.

Le bilan est de **100 paquets envoyés et 98 reçus**, soit **2 requêtes perdues**. Le critère du TP, qui accepte au maximum deux pertes, est donc respecté. Le ping était envoyé toutes les 0,2 seconde.

Ce test m'a permis de vérifier la bascule réelle, au lieu de m'arrêter à la création du bond. Les états complets avant, pendant et après la coupure sont conservés dans les fichiers de preuves.

5. Isoler le trafic transport et choisir sa route

J'ai créé **bond0.42** sur srv-core et **enp0s9.42** sur srv-edge, avec le VLAN 42. Le ping entre **10.42.42.10 et 10.42.42.11** fonctionne. J'ai également capturé le trafic avec tcpdump : on y lit le marquage **vlan 42**, ce qui confirme que les paquets sont bien étiquetés.

```
VLAN 42 - CAPTURE TCPDUMP ENREGISTREE
2026-11-02 20:47:10 CET

$ ip -d link show bond0.42
vlan protocol 802.1Q id 42

Extrait de tcpdump -nei bond0 :
20:24:38.603358 08:00:27:4d:a7:19 > 08:00:27:4b:0d:07, ethertype 802.1Q (0x8100)
, length 102: vlan 42, p 0, ethertype IPv4 (0x0800), 10.42.42.10 > 10.42.42.11:
ICMP echo request, id 3, seq 1, length 64

$ ping -c 1 10.42.42.11
PING 10.42.42.11 (10.42.42.11) 56(84) bytes of data.
64 bytes from 10.42.42.11: icmp_seq=1 ttl=64 time=1.36 ms

--- 10.42.42.11 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.364/1.364/1.364/0.000 ms
_
```

Figure 5 — Interface VLAN, extrait de la capture tcpdump et ping de contrôle.

J'ai ensuite créé la table de routage **transport**, numéro 142. Une règle de priorité 100 sélectionne cette table quand l'adresse source est **10.42.42.10**. Le trafic utilise alors **10.42.42.11 comme passerelle sur bond0.42**. Sans cette adresse source, la route par défaut reste sur **enp0s3**, via le NAT.

```
ROUTAGE PAR SOURCE - CONTROLE EN DIRECT
2026-11-02 20:40:02 CET

$ ip rule show
0:      from all lookup local
100:    from 10.42.42.10 lookup transport proto static
32766:  from all lookup main
32767:  from all lookup default

$ ip route show table transport
default via 10.42.42.11 dev bond0.42 proto static metric 400

$ ip route get 8.8.8.8 from 10.42.42.10
8.8.8.8 from 10.42.42.10 via 10.42.42.11 dev bond0.42 table transport uid 0
    cache

$ ip route get 8.8.8.8
8.8.8.8 via 10.0.2.2 dev enp0s3 src 10.0.2.15 uid 0
    cache
```

Figure 6 — Règle par adresse source et comparaison des deux routes.

Les profils NetworkManager conservent ces réglages après redémarrage. Le contrôle `ip route get` prouve le choix de route sur srv-core ; il ne prouve pas, à lui seul, qu'un accès Internet à travers srv-edge fonctionne.

6. Automatiser l'archivage

J'ai installé le script d'archivage fourni dans les annexes. Il copie la télémétrie, remet le CSV à zéro, compresse la copie et supprime les anciennes archives. Le mode strict, le verrou `flock`, le nettoyage par trap et les messages envoyés à journald rendent son comportement plus facile à suivre.

J'ai testé les trois situations demandées. Pour ne pas fausser le test de source manquante, j'ai arrêté temporairement le collecteur et les déclencheurs, puis tout réactivé une fois le CSV restauré.

Situation testée	Résultat observé
Source disponible, aucun verrou	**0 : succès**
Une autre instance détient le verrou	**2 : exécution refusée**
Fichier source temporairement absent	**3 : prérequis manquant**

```
ARCHIVAGE - TESTS ENREGISTRES ET TIMER ACTIF
2026-11-02 20:47:12 CET

Codes retour mesures :
succes=0
verrou=2
source_absente=3

$ systemctl is-enabled helios-archive.timer
enabled
$ systemctl show helios-archive.timer (NEXT / LAST)
NextElapseUSecRealtime=Tue 2026-11-03 02:30:40 CET
LastTriggerUSec=Mon 2026-11-02 18:32:44 CET

Test de rattrapage enregistre :
Echeance=2026-11-02 20:31:36
Timer arrete avant echeance puis reactive apres.
nov. 02 20:31:40 srv-core.codemax.lan helios-timer-observation[2421]: rattrapage
-observe
Result=success
ExecMainStatus=0
```

Figure 7 — Codes retour, timer activé, prochaine exécution et test de rattrapage.

Le timer planifie l'archivage à **02 h 30**, avec une précision d'une minute, un délai aléatoire maximal de 120 secondes et **Persistent=yes**. L'exécution peut donc être légèrement décalée par rapport à 02:30:00.

Pour vérifier le rattrapage, j'ai utilisé un timer temporaire : je l'ai arrêté avant son échéance, puis relancé après. L'échéance était à **20:31:36** et l'exécution a été enregistrée à **20:31:40**, avec un résultat réussi. Ce test simule l'indisponibilité du timer ; je n'ai pas éteint toute la VM pour cet essai. Le timer temporaire a ensuite été retiré.

Le script des annexes présente une limite : son verrou empêche deux archivages simultanés, mais le collecteur n'utilise pas ce verrou. La copie puis la remise à zéro du CSV ne sont donc pas atomiques face à une écriture concurrente. De même, la purge avec `-mtime +14` raisonne en jours entiers, et non à la seconde près.

7. Comparer cron et systemd, puis tester le bonus

J'ai conservé les observations de la tâche cron de test et des unités systemd pour comparer leur fonctionnement.

Point comparé	Ce que j'ai constaté avec cron	Ce que j'ai constaté avec systemd
Échéance manquée	Je n'ai pas mesuré de rattrapage avec cron.	Le timer de test a rattrapé son échéance après réactivation.
Journaux	La commande de test utilise logger pour laisser une trace.	Je retrouve les messages du script et l'historique du service avec journalctl.
Dépendances	L'entrée testée décrit une commande et un horaire.	Le timer désigne son service, qui vérifie la présence du CSV.
Ressources	Aucune limite propre à l'entrée n'a été ajoutée.	Le service d'archivage applique Nice 10 et une classe E/S idle.
Répartition de charge	L'horaire testé est fixe.	Le timer prévoit un décalage aléatoire maximal de 120 s, sans mesure statistique sur plusieurs jours.
Prochaine exécution	Elle se déduit de l'expression de planification.	list-timers affiche directement NEXT et LAST.

Pour ce besoin, j'ai retenu le timer : je peux suivre son état et ses exécutions avec les mêmes outils que les autres services. J'ai retiré les tâches cron temporaires pour éviter de programmer deux fois l'archivage.

Pour le bonus, j'ai activé **helios-inbox.path**, qui surveille les changements dans **/srv/helios/inbox**. Le dépôt d'un fichier déclenche bien un nouvel archivage. L'unité utilise PathChanged pour éviter une relance en boucle tant qu'un fichier reste présent.

```
BONUS - SURVEILLANCE DU DOSSIER INBOX
2026-11-02 20:47:13 CET

$ systemctl is-active helios-inbox.path
active
$ grep -E "^(PathChanged|Unit)=" helios-inbox.path
PathChanged=/srv/helios/inbox
Unit=helios-archive.service

$ touch /srv/helios/inbox/capture-rapport.csv
$ journalctl -t helios-archive -n 4
Nov 02 20:47:13 srv-core.codemax.lan helios-archive[3601]: demarrage
Nov 02 20:47:13 srv-core.codemax.lan helios-archive[3611]: archive /var/lib/heli
os/archives/telemetry-20261102-204713.csv.gz (225 lignes)
Nov 02 20:47:13 srv-core.codemax.lan helios-archive[3615]: purge : 0 archive(s)
supprimee(s)
Nov 02 20:47:13 srv-core.codemax.lan helios-archive[3617]: fin d execution rc=0
_
```

Figure 8 — Surveillance active du dossier inbox et derniers messages d'archivage.

Le fichier déposé sert de déclencheur : le service archive toujours la télémétrie, pas le contenu du fichier de test.

8. Bilan et fichiers remis

À la fin du TP, le collecteur est actif et démarre automatiquement. Son redémarrage après un arrêt forcé fonctionne, ses limites sont appliquées et son isolation a été renforcée. La coupure d'un lien métier provoque bien la bascule sur le second. Les échanges sur le VLAN 42 et les règles de routage restent présents après redémarrage.

L'archivage renvoie les codes attendus, son timer est activé et le déclenchement par dépôt de fichier fonctionne. Le contrôle de srv-edge confirme son adressage et sa communication avec srv-core sur le VLAN transport.

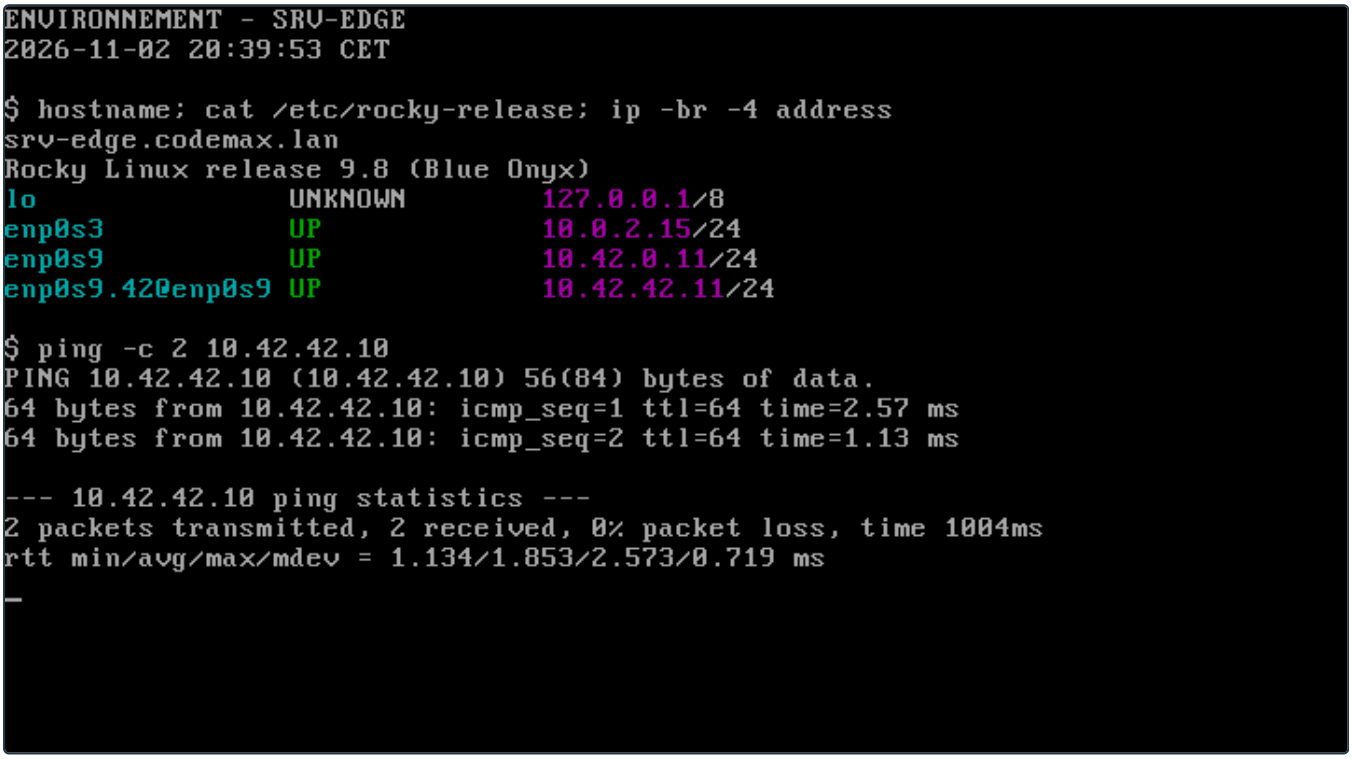


Figure 9 — Adressage de srv-edge et communication avec srv-core sur le VLAN 42.

Ce travail m'a surtout permis de relier chaque configuration à un test concret : arrêter le collecteur, couper un lien, retirer temporairement une source ou laisser passer une échéance. Une commande qui se termine sans erreur ne suffit pas toujours à vérifier le résultat attendu.

Le dossier remis contient le **rapport en PDF**, cette version Markdown, les scripts et unités dans **configs/**, ainsi que les captures et sorties de commandes dans **captures/**. Les graphiques SVG permettent aussi de consulter le détail des démarrages.

Les captures ont été prises après les tests, en affichant les résultats enregistrés ou en relançant les contrôles indiqués. Les mots de passe, clés privées et disques des VM ne sont pas inclus. Les deux machines restent démarrées pour permettre une démonstration.